LinkedIn Post Generator

End-to-end capstone: app, Azure infra, CI/CD, observability

Next.js · FastAPI · PostgreSQL · OpenAI · Terraform · GitHub Actions

Author: Jemine Mene-Ejegi

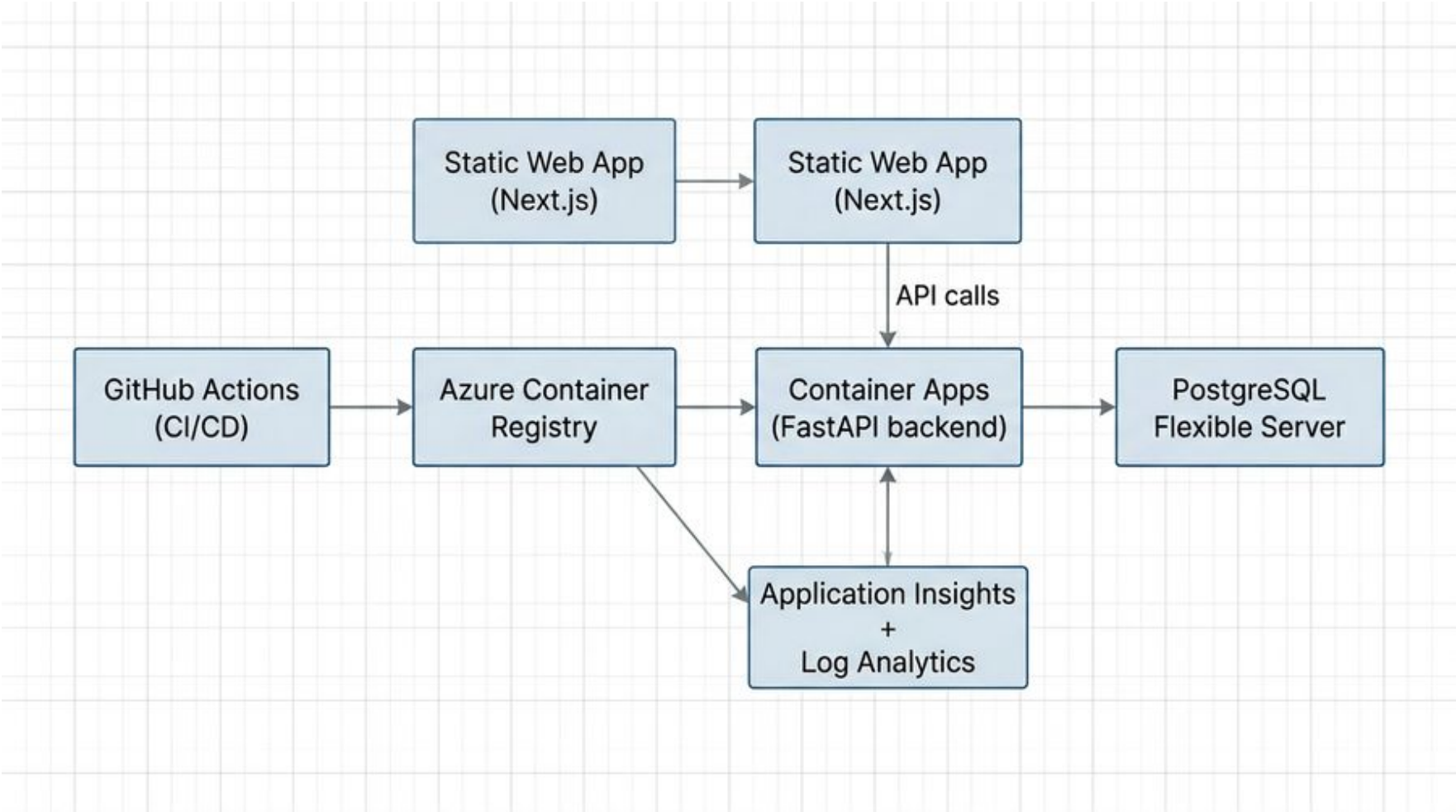
What this system is

- Full-stack product that drafts LinkedIn posts from user input, optional article URLs, and tone/audience settings.
- Optional writing-style learning: user supplies sample posts; backend builds a compact profile via OpenAI.
- History of generations stored in PostgreSQL; users can review and delete past items.
- Production deployment on Azure with infrastructure-as-code (Terraform) and one GitHub Actions pipeline.

User-facing capabilities

- Generate: topic, audience, tone, optional URL (SSR-safe scrape), style mode when a profile exists.
- Style import: paste ≥3 prior posts (--- separator); profile drives faithful / improve / off modes.
- History: paginated list and delete; API mirrors these operations for the static frontend.
- Optional Clerk sign-in in the UI when NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY is set at build time.

High-level architecture



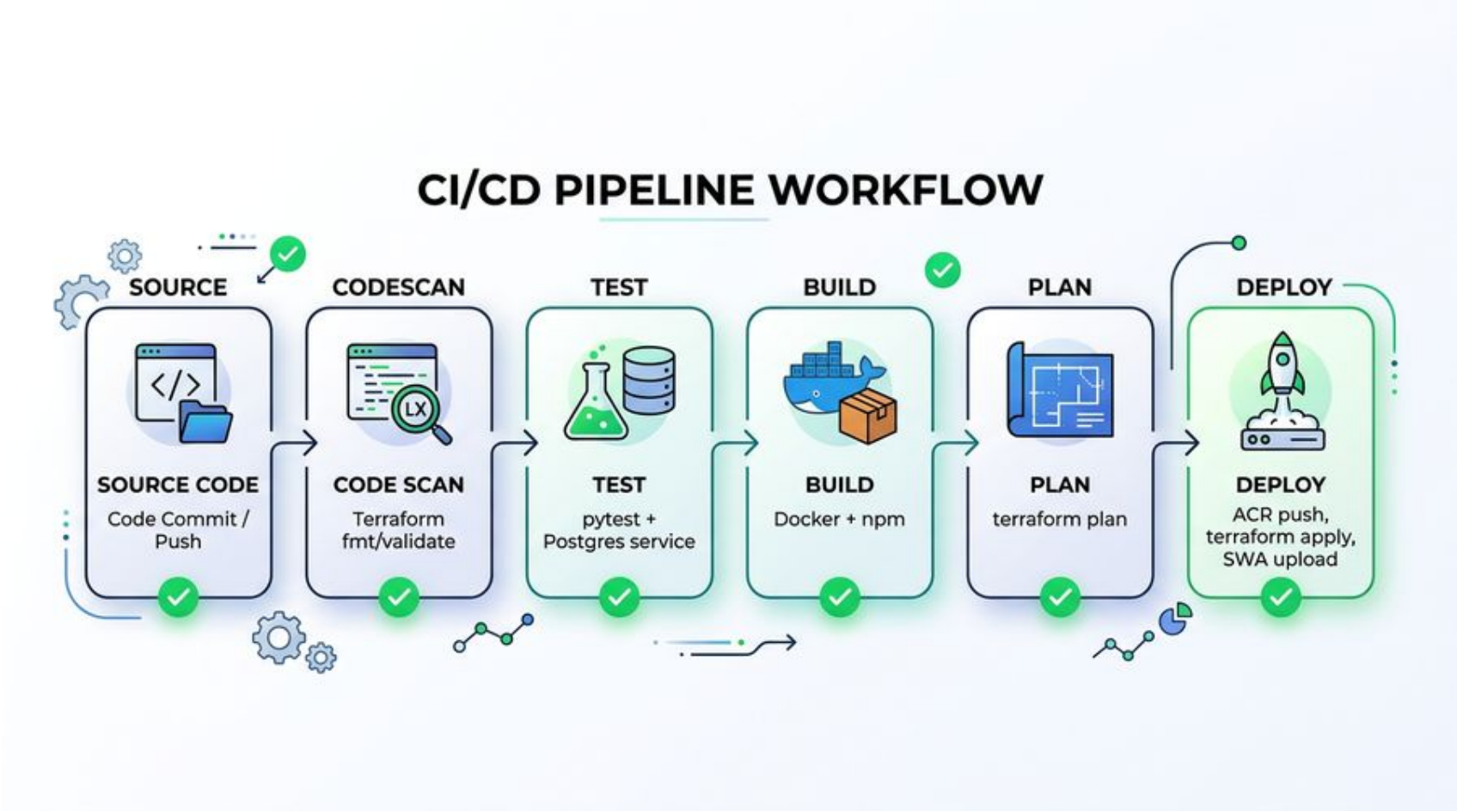
Technology stack

- Frontend: Next.js (static export), Application Insights JS for browser telemetry, optional Clerk.
- Backend: FastAPI, SQLAlchemy, OpenAI SDK, httpx; OpenTelemetry via azure-monitor-opentelemetry when configured.
- Data: Azure Database for PostgreSQL Flexible Server (primary store for posts + style profile).
- Images: Azure Container Registry; runtime: Azure Container Apps with HTTP scaling rules.

Azure resources (Terraform)

- Resource group + tags (project, environment, managed_by = terraform).
- ACR, Container Apps Environment + single backend app (secrets: DATABASE_URL, OPENAI_API_KEY).
- PostgreSQL Flexible Server + database linkedin_gen.
- Static Web App (Free tier) for exported frontend; Log Analytics + workspace-based Application Insights.
- Diagnostic settings: Container App, Static Web App, Postgres → Log Analytics.
- Portal dashboard (summary) + Application Insights workbook (KQL charts); metric alerts if ALERT_EMAIL set.

CI/CD pipeline



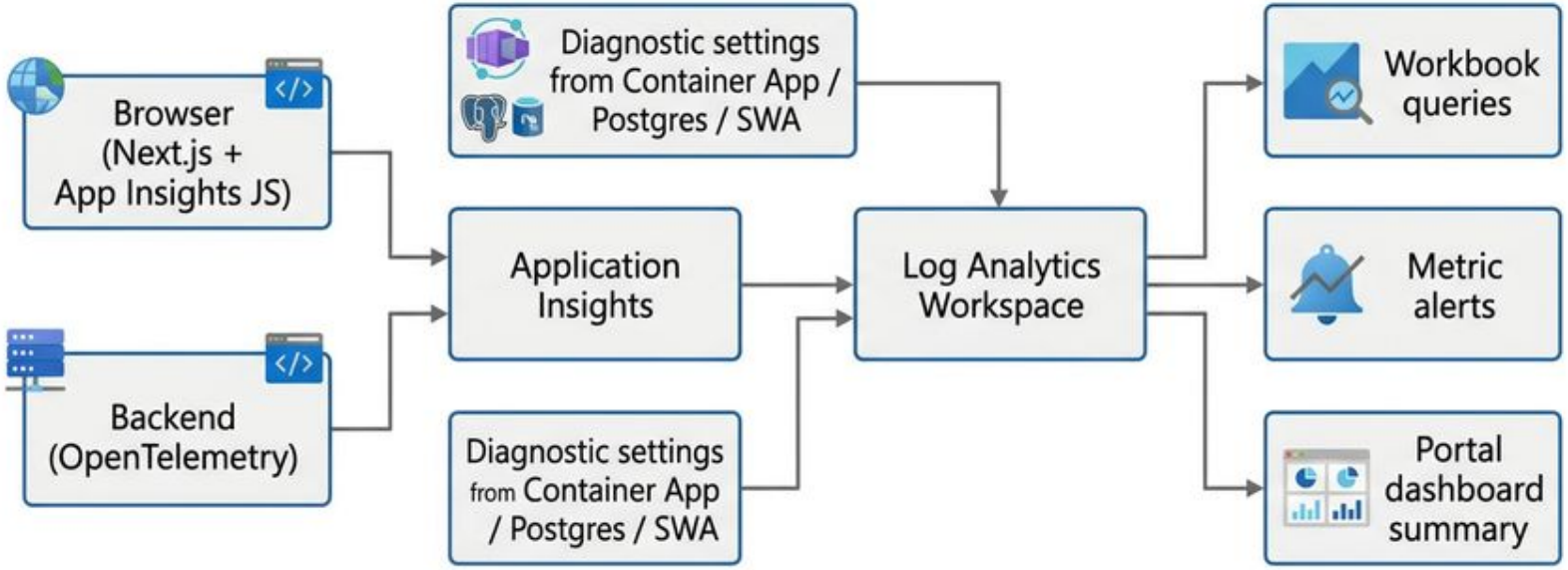
Bootstrap & automation

- scripts/bootstrap.py: remote Terraform state (storage account + container), Azure AD app + federated credential for GitHub OIDC.
- Grants subscription/RG roles needed for CI; writes .deploy.auto.json and sets GitHub repo secrets (Azure IDs, OpenAI, DB password, TF backend, optional
- ALERT_EMAIL).
- scripts/destroy.py: terraform destroy; --include-bootstrap removes GitHub secrets and state RG (requires gh auth).

Security & configuration

- CORS: permissive during apply; pipeline final step sets ALLOWED_ORIGINS to deployed Static Web App URL.
- TrustedHostMiddleware on API; scraper blocks private IPs / SSRF patterns.
- Security headers via staticwebapp.config.json on the exported site.
- Secrets: OpenAI + DB password in Container App secrets and GitHub Actions secrets; bootstrap script wires OIDC + remote state.

Observability — data flow



Telemetry detail (concise)

- Custom metrics (examples): linkedin_generator.generation.*, .style_import.*, .openai.*, .scrape.* — dimensions: tone, style_mode, source_type.
- Frontend events: e.g. frontend_generate_submitted / succeeded / failed for funnel views in Log Analytics.
- Ops workbook: KQL on AppRequests, AppMetrics, AppEvents, AzureMetrics (API volume, latency, failures, CPU/memory).
- Portal dashboard: lightweight Markdown summary linking workbook, App Insights, alerts, and resource names.

API surface (backend)

- POST /api/generate — generate and persist a post.
- GET /api/history — paginated history; DELETE /api/history/{id}.
- POST /api/style/import — build profile; GET /api/style/profile — fetch profile.
- GET /health/live, /health/ready — liveness and DB readiness (used by probes and CI smoke tests).

Local development & quality

- docker compose for Postgres + API; frontend: npm run dev against NEXT_PUBLIC_API_URL.
- Backend tests: pytest with PYTHONPATH=backend; CI uses Postgres 16 service container.
- Terraform: fmt -check, validate, remote state; destroy.py for teardown (optional --include-bootstrap).

Key takeaways

- Single pipeline delivers app + infra: build, push, apply, configure static site, validate health.
- Observable by design: workspace-based App Insights, custom OTel metrics, browser events, workbook + optional alerts.
- Production-minded defaults: secrets isolation, CORS tightening, trusted hosts, SSRF-hardened scraping.
- Repository: README documents outputs, bootstrap, troubleshooting, and diagram assets used in this deck.

Thank you

Questions? See README.md in the repo for deep links, Terraform outputs, and ops runbooks.

— Jemine Mene-Ejegi